

CastUp Architecture & Technology Overview

Welcome to the technical overview of **CastUp**. This document breaks down the technology stack, backend platforms, data storage mechanisms, application data flow, APIs, and AI integrations that power the platform.

1. Technology Stack

Frontend (Client-Side)

- Framework:** React.js (built with Vite for fast performance)
- Routing:** React Router DOM (client-side routing)
- Styling:** Tailwind CSS (utility-first CSS) + Custom CSS variables (`index.css`) for theming
- Animations:** Framer Motion (page transitions, modal animations)
- Icons:** Lucide React
- State Management:** React Context API (`RealAuthContext.jsx` manages user sessions, notifications, and global data)

Backend (Server-Side)

- Runtime:** Node.js
- Framework:** Express.js (RESTful API architecture)
- Authentication:** JWT (JSON Web Tokens) with `bcrypt` for password hashing
- File Uploads:** Multer (handling multipart/form-data)

2. Platforms & Data Storage

Database Platform: Supabase (PostgreSQL)

- How data is stored:** The core relational database is hosted on **Supabase** (running on AWS regions). We connect to it using the `pg` (`node-postgres`) package rather than the Supabase client directly, writing raw SQL queries or using a custom query builder.
- Core Entities/Tables:**
 - `users`: Stores profiles, roles (actor, director, admin), and auth credentials.
 - `casting_calls`: Stores job postings and audition requests.
 - `applications`: Links users to casting calls (many-to-many relationship).
 - `works`: Stores portfolio items and uploaded work.
 - `notifications`: Manages in-app alerts and direct messages.
 - `connections`: Tracks professional connections between users (like LinkedIn).

Cloud Storage: Cloudinary

- Purpose:** Real-time media storage.
- How it works:** When a user uploads a profile picture, or applies to a job with a PDF resume/portfolio, the Node.js backend receives the file via `multer`, buffers it, and securely streams it to **Cloudinary**. Cloudinary returns a secure URL (e.g., `res.cloudinary.com/...`), which is then saved into the Supabase database.

3. How Data is Shown in the App (Data Flow)

- Initialization:** When the app loads, `RealAuthContext.jsx` checks for a secure token in `localStorage`.
- Hydration:** If authenticated, it triggers `refreshPlatformData()`, making parallel API calls (`Promise.all`) to the backend to fetch:
 - All Jobs (Casting Calls)
 - Notifications
 - User Connections & Stats
 - User Profile Data
- Distribution:** This data is stored in React State within the Context Provider and passed down to individual pages via a custom hook (`useAuth()`).
- Rendering:** Pages like `Explore.jsx` or `FindWork.jsx` consume this data and map over it to render UI cards.
- Real-time feel:** For things like `Inbox.jsx`, the frontend polls the notifications endpoint (`setInterval` every 30s) to simulate real-time message updates without requiring heavy WebSockets.

4. APIs & Endpoints Used

The backend exposes a REST API consumed uniformly via `axios` (`api.js` frontend service).

- Auth APIs** (`/api/auth/*`):
 - `POST /register`, `POST /login`, `GET /profile`, `PUT /profile`
- Casting APIs** (`/api/casting/*`):
 - `GET /`, `POST / (Create job)`, `POST /:jobId/apply` (Submit application)
- Admin APIs** (`/api/admin/*`):
 - Gated by admin middleware; manages user deletion, global metrics, and `contact_messages`.
- Media APIs** (`/api/files/*`):
 - `POST /upload` (Bridging to Cloudinary)
- Notification APIs** (`/api/users/notifications/*`):
 - `GET /`, `POST /send`, `DELETE /:id`

5. AI Integration (Google Gemini)

We utilize the **Google Gemini API** (@google/generative-ai SDK) to power intelligent features within the platform. The backend acts as a secure proxy (`aiController.js`) to protect the `GEMINI_API_KEY`.

- AI Assistant Chat:** A conversational bot (`AIChatBot.jsx`) that helps users navigate the film industry. The backend maintains conversation history and falls back gracefully through different internal models (e.g., `gemini-1.5-flash`, `gemini-1.5-pro`) to ensure high availability.
- AI Casting Director:** Allows users to type natural language prompts (e.g., "Find me a strong male lead under 30"). The AI processes the prompt and filters through available talent and job data.
- AI Location Tracker:** Suggests optimal filming locations based on prompt engineering using Gemini's extensive knowledge base.

Summary of the Full Journey

When someone applies for a job: They click "Apply" in the React **Frontend**, the file is sent using **Axios** to the **Express Backend**. The backend uploads the resume to **Cloudinary**, gets the media URL, and stores an `applications` record in **Supabase SQL**. A notification is generated in the DB, which the Casting Director's frontend fetches upon their next poll, illuminating their **Lucide Icon** bell red.